MOVIE REVIEW SENTIMENT ANALYSIS

Using Natural Language Processing and Machine Learning

Minor Project Report

Prepared by

Harsh Tandel

B.Tech – Electronics Engineering

Veermata Jijabai Technological Institute (VJTI), Mumbai

Project Technologies

Python • NLTK • Scikit-learn • TF-IDF • Linear SVM • Streamlit

1. Introduction

Sentiment analysis is a Natural Language Processing (NLP) task used to determine the emotional polarity expressed in text. In the context of movie reviews, sentiment analysis can be used to classify reviews into categories such as positive or negative. This project develops a machine-learning-based movie review sentiment analysis system using the NLTK Movie Reviews dataset, text preprocessing, TF-IDF feature extraction, and supervised classification models.

The project also extends the machine learning workflow into an interactive Streamlit application. A user can enter a movie review through the application, after which the text is preprocessed, transformed into TF-IDF features, and passed to the trained classifier to obtain a sentiment prediction.

2. Objective

• To preprocess movie-review text using fundamental NLP techniques.

• To convert textual reviews into numerical features using TF-IDF.

• To train and compare multiple supervised machine learning classifiers.

• To optimize a Linear Support Vector Machine (SVM) using unigram and bigram TF-IDF features.

• To evaluate the final model on a held-out test set.

• To develop an interactive Streamlit application for real-time sentiment prediction.

3. Dataset

The project uses the NLTK Movie Reviews dataset. The dataset contains 2,000 movie reviews, with 1,000 reviews labelled positive and 1,000 labelled negative. The balanced dataset was used for supervised binary sentiment classification. An 80:20 train-test split was used, resulting in 1,600 training examples and 400 held-out test examples.

The dataset was explored before modelling to inspect its structure, sentiment distribution, review lengths, and missing values. These exploratory checks were used to understand the data before preprocessing and model training.

4. Methodology

The complete workflow consists of dataset exploration, text preprocessing, TF-IDF feature extraction, model training, model comparison, optimization, evaluation, and deployment through Streamlit.

Stage	Description
Movie Reviews	Load labelled positive and negative reviews.
Exploratory Data Analysis	Inspect dataset structure, class distribution, review lengths, and missing values.
Text Preprocessing	Lowercasing, filtering alphabetic tokens, stopword removal, and lemmatization.
TF-IDF	Convert cleaned text into numerical feature vectors.
Train/Test Split	Use an 80:20 stratified split.
Model Training	Train Linear SVM, Logistic Regression, and Multinomial Naive Bayes.
Optimization	Use up to 10,000 TF-IDF features with unigrams and bigrams and sublinear TF.

Stage	Description
Evaluation	Measure classification accuracy and inspect classification performance.
Deployment	Use the saved model and vectorizer in a Streamlit application.

5. Text Preprocessing

The raw reviews are transformed into cleaner text before feature extraction. The preprocessing function converts text to lowercase, splits it into words, retains alphabetic tokens, removes English stopwords, and applies WordNet lemmatization.

Processing sequence: Raw Review → Lowercasing → Alphabetic-token filtering → Stopword removal → Lemmatization → Cleaned Review

This preprocessing reduces unnecessary variation in the input text and produces a consistent representation for the subsequent TF-IDF vectorization step.

6. TF-IDF Feature Extraction

Term Frequency–Inverse Document Frequency (TF-IDF) is used to represent the cleaned reviews numerically. The baseline experiment used TF-IDF with a maximum of 5,000 features. The optimized configuration increased the feature limit to 10,000 and included both unigrams and bigrams with sublinear term-frequency scaling.

Parameter	Final configuration
Maximum features	10,000
N-gram range	(1, 2)
Sublinear TF	True

7. Machine Learning Models

Three baseline supervised classifiers were evaluated using the TF-IDF representation: Linear Support Vector Machine, Logistic Regression, and Multinomial Naive Bayes. An optimized Linear SVM was subsequently trained using the expanded unigram-and-bigram TF-IDF representation.

Model	Accuracy
Linear SVM	83.50%
Logistic Regression	82.25%
Multinomial Naive Bayes	79.50%
Optimized Linear SVM	**85.50%**

The optimized Linear SVM achieved 85.50% accuracy on the 400-review held-out test set used in the experiment.

8. Final Model and Implementation

The final trained classifier is a Linear Support Vector Classifier implemented using scikit-learn's SVC with a linear kernel. The trained model was serialized with Joblib as **sentiment_model.pkl**. The corresponding

TF-IDF vectorizer was saved as **tfidf_vectorizer.pkl**. Keeping the model and vectorizer together allows the application to reproduce the same feature representation during inference.

The preprocessing logic is maintained in **src/preprocessing.py**, while the Streamlit interface and prediction pipeline are implemented in **app/app.py**.

9. Streamlit Application

An interactive Streamlit application was developed as the deployment component of the project. The application provides a text area where a user can enter a movie review. When the Analyze Sentiment button is selected, the application preprocesses the review, transforms it using the saved TF-IDF vectorizer, and passes the resulting feature vector to the saved SVM model.

The application reports the predicted class as either positive or negative. It also identifies the model and feature representation used by the application.

10. Results and Observations

The baseline Linear SVM achieved 83.50% accuracy, while Logistic Regression achieved 82.25% and Multinomial Naive Bayes achieved 79.50%. Expanding the TF-IDF representation to 10,000 features and including unigrams and bigrams improved the Linear SVM result to 85.50% on the held-out test set.

Manual testing through the Streamlit application demonstrated that straightforward positive and negative reviews can be classified. A strongly negative review containing criticism such as excessive violence, gratuitous gore, and weak storytelling was also tested and was incorrectly classified as positive. This is a useful error-analysis observation: the classifier relies on learned statistical word and phrase patterns and can struggle with complex contextual language and nuanced criticism.

11. Limitations

• The final accuracy is measured on a particular held-out test set and does not guarantee correct classification of every new review.

• The training dataset is limited to the NLTK Movie Reviews corpus.

• Complex context, sarcasm, negation, and nuanced criticism may be difficult for a conventional TF-IDF-based classifier.

• The model is a classical machine learning system and does not provide the deeper contextual representation available in modern transformer-based language models.

12. Conclusion

This project demonstrates an end-to-end movie review sentiment analysis pipeline using Natural Language Processing and machine learning. The workflow covers data exploration, preprocessing, TF-IDF feature extraction, model comparison, optimization, evaluation, model serialization, and deployment through a Streamlit interface.

Among the evaluated configurations, the optimized Linear SVM with TF-IDF unigram and bigram features achieved 85.50% accuracy on the held-out test set. The resulting model was integrated into an interactive application that allows users to submit movie reviews and receive sentiment predictions.

13. Future Scope

• Use larger and more diverse movie-review datasets.

• Perform systematic hyperparameter tuning and cross-validation.

• Experiment with transformer-based models such as BERT-family architectures.

• Add confidence or probability-style output where supported by the selected model.

• Extend the system to multilingual movie reviews.

• Develop additional visual analytics for sentiment trends and review characteristics.

14. Project Structure

```
Movie_Sentiment_Analysis/
■■■ app/
■ ■■■ app.py
■■■ models/
■ ■■■ sentiment_model.pkl
■ ■■■ tfidf_vectorizer.pkl
■■■ src/
■ ■■■ preprocessing.py
■■■ results/
■ ■■■ model_comparison_results.csv
■■■ screenshots/
■■■ report/
■■■ README.md
■■■ requirements.txt
```

15. Running the Application

Install the required packages using the project's requirements file and start the Streamlit application from the project root:

```
pip install -r requirements.txt
streamlit run app\app.py
```

The final environment used scikit-learn 1.6.1 to match the version used when the serialized model was created.